

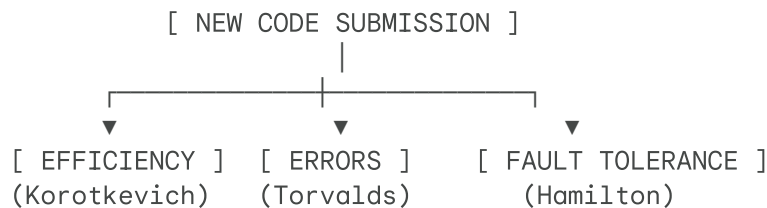
CONTRIBUTING TO SOVEREIGN SHARDS

Welcome to Sovereign Shards. This document defines the non-negotiable engineering standards, cognitive workflow partitions, style conventions, and testing protocols for human developers and autonomous AI contributors.

Sovereign Shards is designed to execute cleanly in highly resource-constrained, air-gapped environments. These guidelines are strict and are programmatically enforced at the commit boundary.

1. THE FIVE ENGINEERING MASTERS

All code modifications are statically evaluated against five software engineering paradigms before being merged into `main`. The rules are parsed using AST checking utilities (`core/fivemasters.py`):



1. Korotkevich (Efficiency Invariant)

- **No Redundant Lookups:** Avoid repeated dictionary lookups inside loops. Cache loop values or lookups in local variables.
- **No Empty Collections in Hot Paths:** Do not instantiate empty lists, dictionaries, or sets inside high-frequency call sites.
- **Prioritize Generator Expressions:** Avoid materializing full lists with list comprehensions or `list(filter(...))` when iterating. Use lazy generator expressions to minimize memory spikes:

$$\text{Memory Overhead}_{\text{Generators}} \ll \text{Memory Overhead}_{\text{Comprehensions}}$$

2. Torvalds (Error Handling Invariant)

- **No Bare Exceptions:** Bare `except:` statements are strictly prohibited under any circumstances.
- **No Silent Passing:** Code blocks containing empty pass-through loops or silent catches such as `except Exception: pass` are banned.
- **Mandatory Trace Logging:** Every caught exception must catch an explicit exception class, log the precise traceback or message, and fail closed gracefully.

- **Strict Algorithmic Limits:** Avoid nested iteration loops that exceed $O(n^2)$ time complexity.
- **No Loop Concatenation:** Never use string concatenation using the `+` operator inside loops. Build lists and use `''.join()` for efficient memory allocations.
- **Shallow Function Nesting:** Keep call stacks shallow. Any logical flow nested deeper than 4 call layers must be refactored into flattened pipeline handlers.

4. Hamilton (Fault Tolerance Invariant)

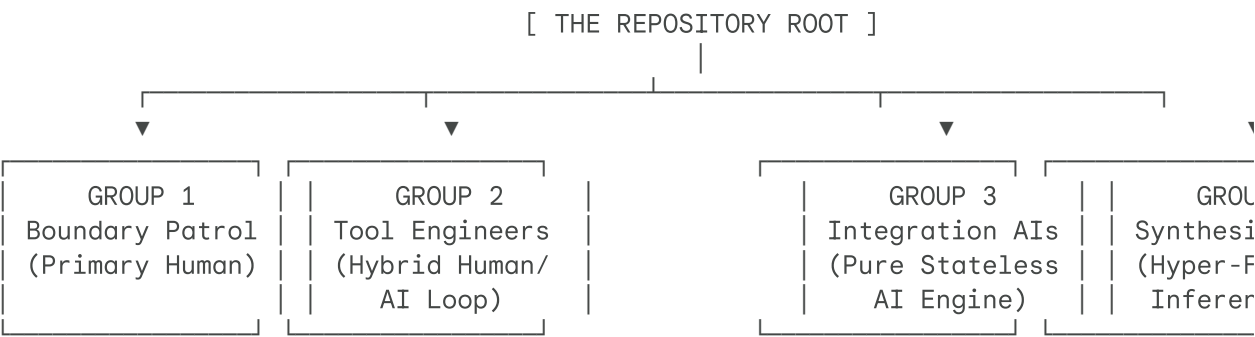
- **Guaranteed Return Paths:** Every function exceeding 5 lines of code must contain an explicit and safe return path that handles unexpected errors or edge case conditions.
- **Defensive Parameter Boundaries:** Validate types, character lengths, and bounds of all incoming arguments at the function entry point before processing logic.

5. Ritchie (Clarity Invariant)

- **Standard Variable Naming:** Functions and variables must strictly use `snake_case` .
- **Standard Class Naming:** Classes must strictly use `PascalCase` .
- **Constant Naming:** Global variables, settings, and constants must use `ALL_CAPS` .
- **Exempt dunder/private names:** Dunder methods (`__init__`) and private helpers (`_leading_underscore`) are naturally exempt.

2. THE DUAL-ENGINE OPERATIONAL WORKFLOWS

To eliminate development drift and prevent token context bloat, human engineers and automated AI agents must operate under a strictly partitioned labor matrix:



Group 1: The Boundary Patrol (Primary Human Engineers)

- **Domain Focus:** Core Framework Architecture, Immutable Safety Rules, Setup Shell Scripts, and System Invariant Handlers.
- **Core Assets:** `app/chat.py` , `setup.bat` , `tools/run/_path_guard.py` .
- **Mandate:** Human leads own the core. They hardcode the rules of engagement, maintain the **Zero-Inference Router** regex blocks, and protect the system against memory expansion or

- **Efficiency Metric:** Maintain active multi-model configurations strictly under the **4.5 GB** collective memory budget.

Group 2: The Tool Engineers (Hybrid Human/AI Developers)

- **Domain Focus:** Extending the dynamic `ToolRegistry` and building isolated, single-purpose utilities.
- **Core Assets:** `tools/run/*.py` , `tools/run/registry.json` .
- **Mandate:** Create standalone scripts capable of executing one system task safely.
- **Efficiency Metric:** Script tools must execute cleanly inside isolated subprocess blocks and return output values within a strict **300ms compute window**.

Group 3: Context Integration Layer (Pure Stateless AI Agents)

- **Domain Focus:** Automated File Ingestion, Data Partitioning, and Multi-Model Swapping.
- **Core Assets:** Ingestion files, task buffer files, and pipeline execution logs.
- **Mandate:** Integrate files and manage model hand-offs. These agents are **structurally blocked from outputting natural conversational language**. They communicate only using valid, parsable JSON frames or precise machine action tokens.
- **Efficiency Metric:** Maintain zero conversational preambles inside system logs or execution files.

Group 4: Synthesis & Validation Layer (Hyper-Focused Inference AIs)

- **Domain Focus:** Local Log Reconciliation and Verification Audits.
- **Core Assets:** Markdown audit logs, change summaries, and delta outputs.
- **Mandate:** Read raw execution logs, reconcile difference deltas between the live Context Log and the active Working Log, and write highly compressed, zero-hallucination markdown summaries for final human sign-off.
- **Efficiency Metric:** Compress multithread logs into dense, single-page verification updates within a maximum 3-hop limit.

3. FILE DEVELOPMENT TEMPLATE

When writing a new tool script within `tools/run/` , use the following standard template to ensure compatibility with the auto-discovery registry:

```
# -*- coding: utf-8 -*-
"""
Sovereign Shards - Standard Tool Module Template.
Author: [Contributor Name/ID]
"""
from __future__ import annotations
import sys
import json
from pathlib import Path

def run(tool_name: str, arguments: list[str]) -> str:
```

```

def run(stdin_payload: str, arguments_payload: dict) -> str:
    """
    Main entry point for tool execution.
    Hamilton Invariant: Defensive boundary validation and explicit return paths.
    """
    try:
        # validate inputs safely
        if not arguments_payload:
            return json.dumps({"ok": False, "error": "Missing payload fields."})

        target_path_raw = arguments_payload.get("path")
        if not target_path_raw:
            return json.dumps({"ok": False, "error": "Target path parameter is em

        # Enforce path containment and escape validations
        target_path = Path(target_path_raw)
        if target_path.is_absolute() or ".." in target_path.parts:
            return json.dumps({"ok": False, "error": "Path containment violation:

        # Ritchie Invariant: clear variable declarations
        result_message = f"Inspected target path: {target_path.as_posix()}"

        return json.dumps({
            "ok": True,
            "output": result_message
        })

    except Exception as exc:
        # Torvalds Invariant: non-silent, explicit error handling
        return json.dumps({
            "ok": False,
            "error": f"Execution failed: {str(exc)}"
        })

if __name__ == "__main__":
    # Ensure standard UTF-8 stream handling for Windows environments
    try:
        sys.stdout.reconfigure(encoding='utf-8', errors='replace')
    except AttributeError:
        pass

    # Read pipeline input arguments
    raw_args = sys.argv[1] if len(sys.argv) > 1 else "{}"
    try:
        parsed_args = json.loads(raw_args)
    except Exception:
        parsed_args = {}

    stdin_data = sys.stdin.read() if not sys.stdin.isatty() else ""

    # Write output to standard stdout
    print(run(stdin_data, parsed_args))

```

4. CODE STYLE & STYLE CONVENTIONS

- **Python Target:** Python 3.10+ (standard support for pattern matching, union types, and f-strings).
- **First Line Invariant:** Every active Python script in this repository must declare `from __future__ import annotations` as its first active line of code.
- **Strict Type Annotations:** All public and private function parameters and return signatures must be explicitly typed using standard type hinting syntax.
- **Directory Partitioning:** Core engine files are isolated inside `app/`. Project extensions and web wrappers are restricted to `projects/`. Tests are placed inside `tests/`, and temporary playpens live inside `scratch/` or `examples/`.

5. TESTING & COMMIT WORKFLOW

1. Local Pre-Commit Checks

Before committing any changes to the code tree, run the complete local unit test suite to verify codebase health:

```
python -m unittest discover tests/
```

2. Static Analysis Linting

All modifications must compile cleanly under the strict Ruff configuration criteria. Run the linter to clean up syntax, trailing characters, and imports:

```
ruff check . --fix
```

3. Commit Message Syntax

We enforce strict commit styling templates to preserve the history of our branch tree. All commit messages must follow this structure:

```
type(G[Group_Number]-[feature_key]): concise structural summary
Signed-off-by: [Your Name / Agent ID] - Core Multi-Model Path
```

Example Commit Message:

```
feat(G2-calc): integrated safe AST calculator parsing to intercept math lookups
Signed-off-by: Agent [Viktor-09-2025] - Core Multi-Model Path
```

[CONTRIBUTING STANDARDS LOCKED: FIVE MASTERS COMPLIANT IMPLEMENTATION DETECTED]

